

III-Semester Examination V III III

Page No. 10



Signature of the student

Roll No.
Signature of the student
Centre No.

INSTRUCTIONS TO CANDIDATE

1. The candidate must read the instructions carefully before attempting the examination.
2. The candidate must write the answers in the answer book provided.
3. The candidate must write the answers in the answer book provided.
4. The candidate must write the answers in the answer book provided.
5. The candidate must write the answers in the answer book provided.
6. The candidate must write the answers in the answer book provided.
7. The candidate must write the answers in the answer book provided.
8. The candidate must write the answers in the answer book provided.
9. The candidate must write the answers in the answer book provided.
10. The candidate must write the answers in the answer book provided.

1

A

```
printf("Simple Interest = %.f\n", si);
```

Lexemes → 7.

· "printf"

· '('

· "Simple Interest = %.f\n"

· ','

· 'si'

· ')'

· ';'.

Tokens → 7

< keyword printf >

< Rbracket (>

< input id >

< comma , >

< identifier si >

< Lbracket) >

< semicolon ; >

B

Lex program to identify comments.

```
%L #include <stdio.h> %L
```

```
%L %L
```

```
%L // ] printf("Write Comment = %s", yylex);
```

```
%L %L
```

```
int main(void) {
```

```
    yylex();
```

}

C.

Front end & Back end.

• Front end:

→ Front end consists of phases which are dependent on source language.

• Lexical Analyser

• Syntax Analyser

• Semantic Analyser

• Intermediate Code Generator.

• Back end

→ Back end consists of phases which are dependent on target code.

• Code Optimiser

• Code Generator.

F. Parse tree

$S \rightarrow AB$ ^{grammar}

$A \rightarrow a$

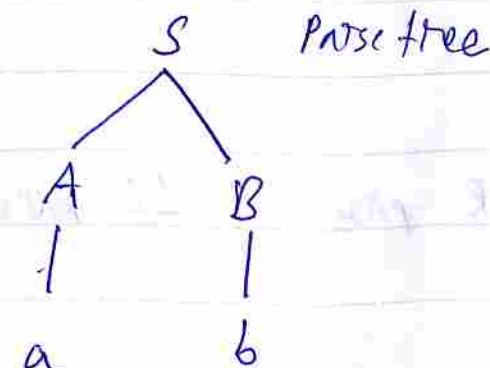
$B \rightarrow b$

String $\rightarrow ab$

$S \rightarrow AB$

$\rightarrow aB$

$\rightarrow ab \checkmark$



The string ab is obtained by grammar parse tree

D1.

follow (1

To construct follow function we need the parser to convert data from one format to another. This helps with the data to be converted.

E1.

LR parser LL parser

• LL Parser is a top down parser
→ we check Left Derivation, left factoring, First & follow
fun. 1 to be tabulated into a table

• LR Parser is a bottom up parser
→ There is also SLR which is Simple LR.
it makes it simpler for the compiler.

2

A1.

Create format for Lex Program.

lex.l → lex → lex.yy.c.

lex.yy.c → gcc → a.out

input → a.out → output.

Lex program generally consists of 3 things.

- | | |
|--------------------------|---------------|
| • Definitions | • Definitions |
| • Rules | • Rules |
| • Nodes | • Code |

Definitions.

- It is enclosed within `%L %Y`
- It contains C codes

`%L`

`#include <stdio.h>`

`%Y`

Rules

- It is enclosed within `%L %Y`
- It contains pattern action pairs with regular expressions

`%L`

`[0-9] print("%d", yylval);`

`%Y`

Code

- It contains additional C code

`int main(void)`

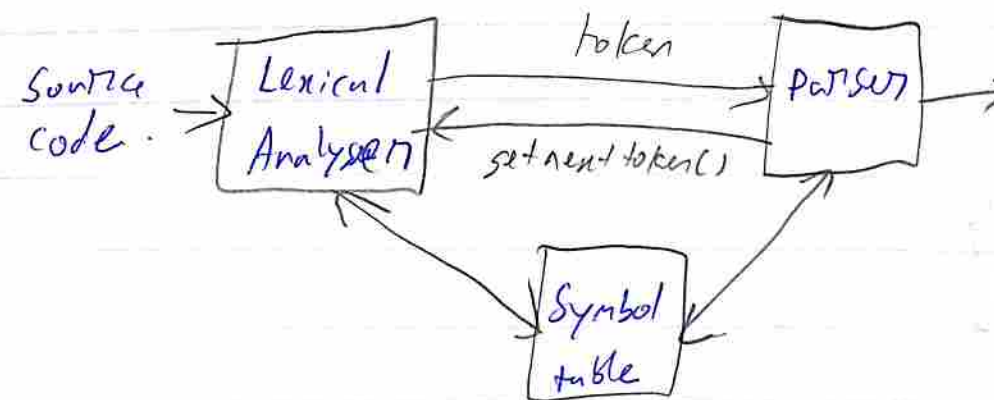
`yylex();`

`y`

B1.

Lexical analysis.

- Lexical analyzer is the first phase of the compiler.
- It is used to break down the source code to tokens.



The lexical analyzer when first given with source code removes all the white space and breaks everything down to Lexemes. These Lexemes are sent to the Symbol table for them to be assigned with its ident tagword, which are now as Tokens. This whole process is monitored by the Rules of Pattern. The generation of tokens is the important part of the lexical analyzer.

C7.

Recursive Descent Parser.

$S \rightarrow AA$

$A \rightarrow aA | b$

First we check for

• Left Recursion

• Left Factoring

SC12.

A();

A();

y

A()2.

if(input == 'a')2 input ++;

A();

y

else2

if(input == 'b')2

input ++;

y

y

D

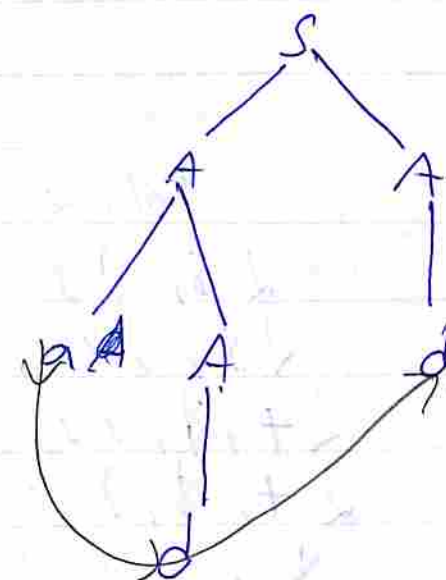
D1.

CLR Parse table

$S \rightarrow AA$

$A \rightarrow aA | d$

A parser is use to
convert data from one
format to another.



'add' string.

SC12

A();

A();

y

$S \rightarrow AA$

$A \rightarrow aA | d$

AD

5 A)

First & Follow.

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' / \epsilon$$

$$F \rightarrow (E) / id$$

	First	Follow
E	{ id, (}	{ \$,) }
E'	{ +, ε }	{ \$,) }
T	{ id, (}	{ +, \$,) }
T'	{ *, ε }	{ +, \$,) }
F	{ id, (}	{ *, +, \$,) }

B)

Top - Down Parser.

- There are two types of parsers, TOP-Down and Bottom-Up Parsers
- In The TOP-Down Parser, it starts at the Root Node then moves to leaf nodes

Recursive Descent Parser.

$$A \rightarrow abC / aBd / aAD$$

$$B \rightarrow Bb / \epsilon$$

$$C \rightarrow d / \epsilon$$

$$D \rightarrow a/b / \epsilon$$

String = aaba

$$B \rightarrow Bb / \epsilon \rightarrow \text{Left Recursion}$$

$$B \rightarrow \epsilon B'$$

$$B' \rightarrow bB' / \epsilon \rightarrow B \rightarrow b / \epsilon$$

A() {

```

    if (input == 'a') {
        input++;
        if (input == 'b') {
            input++;
            C();
        }
    }

```

y
y else {

```

    if (input == 'a') {
        input++;
        B();
        if (input == 'a') {
            input++;
        }
    }

```

y
y
y else {
 if (input == 'a') {
 input++;

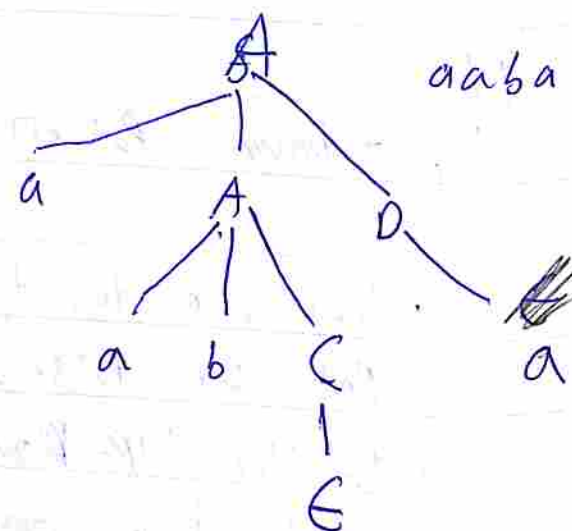
```

        A();
        D();
    }

```

y

y



a a b ε a

⇒ a a b a.

Parse tree
aaba

B() {

```

    if (input == 'b') {
        input++;
    }

```

```

    }
    else {
        return;
    }

```

C() {

```

    if (input == 'a') {
        input++;
    }

```

```

    }
    else {
        return;
    }

```

D() {

```

    if (input == 'a') {
        input++;
    }

```

```

    }
    else {
        if (input == 'b') {
            input++;
        }
    }

```

```

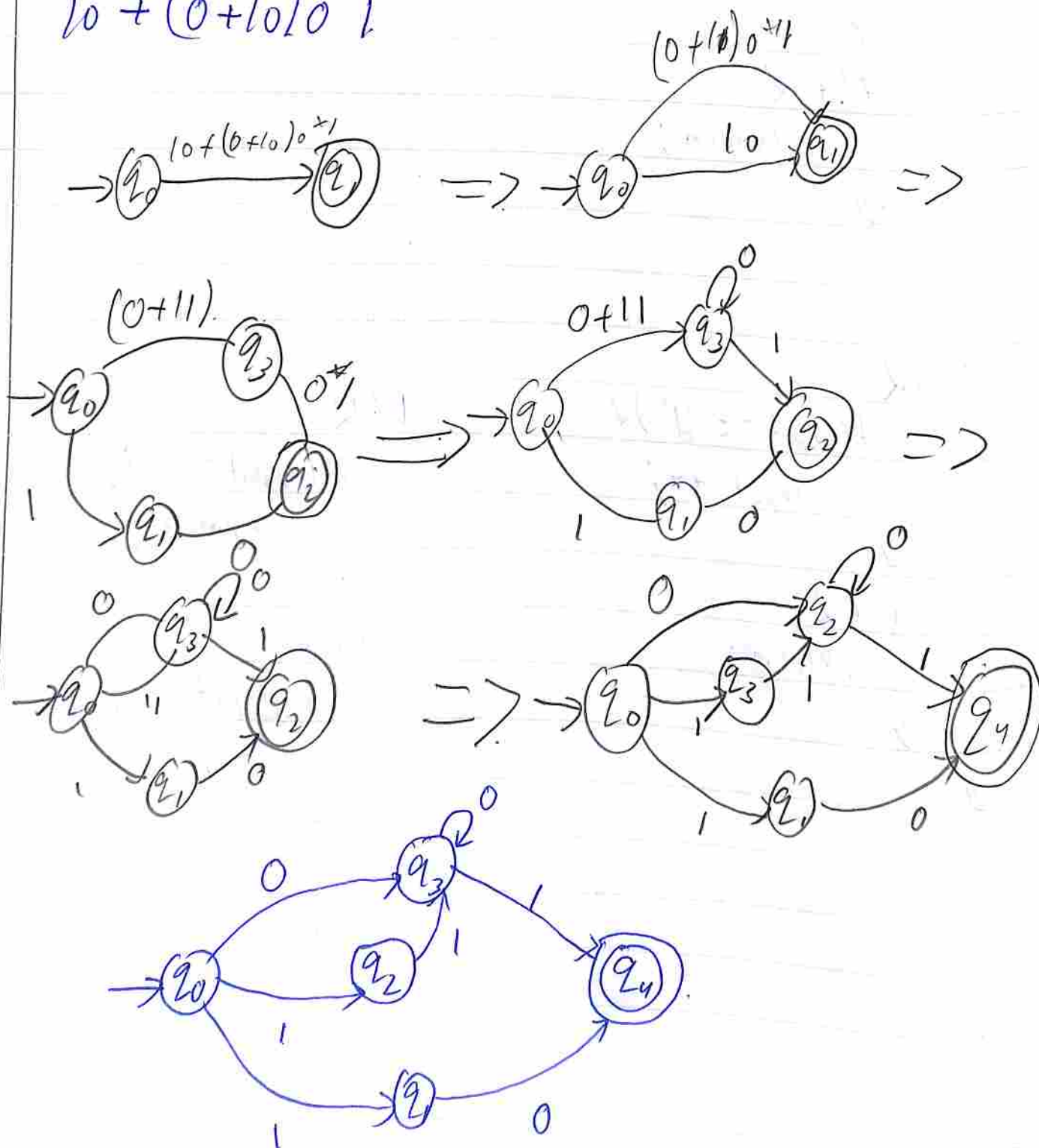
    }
    else {
        return;
    }

```

3 A

Regular expression.

$10 + (0 + 10)0^*1$

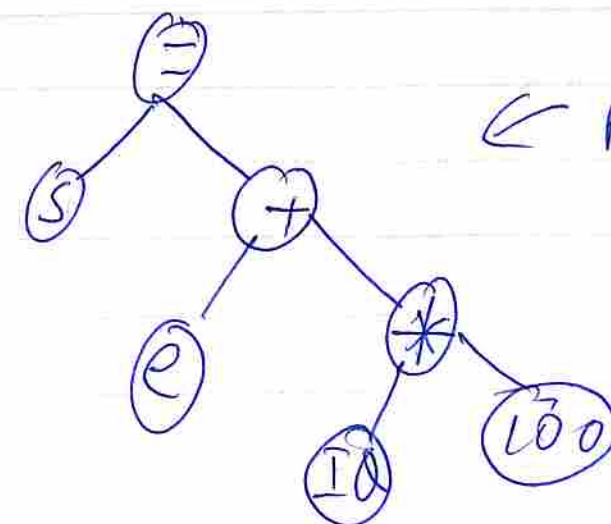


B

Success : = effort + 10 * 100

3 address code.

$s' \quad s' \quad e \quad + \quad '10' \quad * \quad '100'$



← parse tree format

$\langle \text{identity } s \rangle$
 $\langle \text{equals } = \rangle$
 $\langle \text{identity } e \rangle$
 $\langle \text{add } + \rangle$
 $\langle \text{identity } 10 \rangle$
 $\langle \text{mult } * \rangle$
 $\langle \text{number } 100 \rangle$

$T_1 = 10 * 100$
 $T_2 = e + T_1$
 $s = T_2$

Time not there.

18

$$x(n) = A + \sin(2\pi n)$$

Plot

